Explore your Linux system

Activity 1 of 6 | 5 minutes | Kernel and distribution

Name: Muadh Adekunle Monsuru **Student ID:** 20261006026
Date: 14/10/2026 **Partner, if applicable:** _____

In this lab, you will connect operating system concepts to observations from Linux. Complete the activities in order and record your own results. Allow 45-60 minutes, including discussion. Activity 6 is optional.

Before you begin

Use an ordinary user account and a Bash terminal. If necessary, type **bash** first. Keep the same terminal open throughout the lab. Do not use sudo. Output and available commands may differ between machines. If a command is missing, tell your instructor and record the limitation.

Run these commands

```
uname -s
uname -r
cat /etc/os-release
```

uname -s reports the kernel name. **uname -r** reports the running kernel release. **cat** displays a file: /etc/os-release contains distribution identity information.

Record your observations

Kernel name and release:

Linux
7.0.0-34-generic

Distribution name and version, if reported:

PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION="24.04.4 LTS (Noble Numbat)"

Environment: physical computer, virtual machine or container? If unsure, write “unknown”.

Physical Computer

Explain

What is the difference between the kernel and a Linux distribution? Use your output as an example.

The Kernel is the core software that talks directly to the computer's hardware (managing the CPU, memory, and devices). my uname -s and uname -r commands reveal that my system is running the Linux kernel, specifically version 7.0.0-34-generic.

The Distribution (Distro): This is the complete, ready-to-use operating system. It takes the base Linux kernel and bundles it with system tools, a package manager, default applications, and usually a graphical interface. My cat /etc/os-release output shows that my specific distribution is Ubuntu 24.04.4 LTS.

Inspect the CPU topology

Activity 2 of 6 | 8 minutes | Processor packages, cores and logical CPUs

Run this command

```
lscpu
```

This command reports the CPU architecture and topology visible to Linux. Find the fields below. Field labels may differ if your system uses another language. Write “not reported” for missing values.

Record your observations

CPU(s):

8 CPUs

Thread(s) per core:

2 Threads per Core

Core(s) per socket:

4 cores per socket

Socket(s):

1 socket

CPU(s) describes logical CPUs. Thread(s) per core describes hardware execution contexts, rather than the number of software threads an application creates. A virtual machine reports the topology exposed to its guest OS.

Explain and calculate

A hypothetical system has 1 socket, 2 cores per socket and 2 threads per core. How many logical CPUs does it expose? Show your calculation.

Total Logical CPUs = Sockets × Cores per Socket × Threads per Core
1 × 2 × 2 = 4

Why should we distinguish a physical core from a logical CPU?

It is crucial to distinguish between them because they do not offer the same processing power or hardware resource. A physical core is actual hardware with its own dedicated execution engine. A logical CPU is a software construct that allows two instruction threads to share the resources of a single physical core.

If you are using a virtual machine, can this output establish the complete CPU topology of the physical host? Explain.

No. If you are operating inside a virtual machine (VM), the terminal output only shows the virtual hardware topology presented to the guest operating system, not the physical host's actual layout.

Compare memory and storage

Activity 3 of 6 | 7 minutes | RAM, filesystem capacity and caching

Run these commands

```
free -h
df -h .
```

free reports memory and swap information. **df** reports filesystem capacity and usage. In both commands, **-h** requests readable units. The dot in **df -h .** selects the filesystem containing your current directory.

Record your observations

RAM: total, free and available. Include the displayed units.

Total :15Gi used: 7.2Gi free: 4.7Gi shared: 1.2Gi

Filesystem: size, used space and available space. Include the displayed units.

Size: 468G Used: 267G Available: 177G

Filesystem mount point, shown under “Mounted on”:

Mount point: /

Interpret the results

The available-memory value estimates capacity available for additional work, including some reclaimable cached memory. It may therefore differ from the free-memory value. Filesystem figures describe a different resource.

Does 100 GB of available filesystem space mean that the computer has 100 GB of RAM? Explain the distinction.

Filesystem space and RAM are entirely different hardware resources; having 100 GB of available filesystem space does not indicate how much RAM the system has. Filesystem is persistent, long-term storage where operating system, applications, and files reside permanently. RAM (Memory) is volatile, high-speed workspace. It holds only the active data and running applications.

Why can Linux use RAM for caching even when applications may need more memory later?

The kernel automatically uses unoccupied RAM to cache disk data (the page cache), which dramatically speeds up read and write operations. When active applications suddenly need more memory, the kernel simply discards the cached disk data and instantly reallocates that RAM to the application.

Which command would you use to investigate RAM pressure, and which would you use when a filesystem is nearly full?

RAM Pressure: free -h

Filesystem Space: df -h

Create and observe a process

Activity 4 of 6 | 10 minutes | Process identity, waiting and CPU use

Start a process and inspect it

```
sleep 120 &
pid=$!
echo "$pid"
ps -o pid,ppid,stat,pcpu,comm -p "$pid"
```

sleep 120 waits for approximately 120 seconds. **&** starts it in the background. **$!** gives Bash the PID of its most recent background job; **pid=$!** saves it. Do not put spaces around the equals sign.

ps -o selects output columns. **-p "$pid"** selects your process. PID identifies the process, PPID its parent, STAT its state, %CPU its reported CPU use and COMMAND the command name. CPU use is an average, not an instantaneous measurement.

Record your observations before sleep finishes

PID, PPID, STAT, %CPU and COMMAND:

PID: 36520, PPID: 25267, STAT: S %CPU: 0.0 COMMAND: SLEEP

Does the process exist while it is waiting? What evidence supports your answer?

Yes, the process continues to exist while it is waiting.

The process is listed with its assigned Process ID (PID 36520), proving the operating system is still tracking and managing it.

End the process you created

```
kill "$pid"
wait "$pid"
```

Run these in the same terminal while sleep is still active. **kill** sends SIGTERM by default. **wait** waits for this child to terminate and collects its exit status. A “Terminated” message or nonzero status is expected after signal termination. If sleep has already finished, repeat the activity before testing these steps.

Explain

How does waiting with sleep differ from repeatedly checking a condition in a busy-wait loop?

The process pauses and voluntarily gives up the CPU. The operating system places it in a low-power wait state where it consumes zero CPU cycles until the time expires. While on the other hand, polling process remains active on the CPU, continuously running a loop to check if a condition is met. It aggressively consumes processor cycles (often 100% of a core) while doing no productive work.

Which resource can another runnable task use while this process waits?

Another runnable task can immediately use the CPU (processor cycles).

If ps displays only a header, the process may already have ended. This activity demonstrates timed waiting, rather than a device I/O operation.

Create a file and change access

Activity 5 of 6 | 10 minutes | Files, ownership and protection

Create a fresh practice directory and file

```
lab_dir=$(mktemp -d)
cd "$lab_dir"
printf 'Operating systems manage resources.\n' > message.txt
cat message.txt
ls -l message.txt
```

mktemp -d creates a unique temporary directory. **$(...)** captures its path, and **cd** enters it. **printf** produces text; **\n** is a newline. **>** writes to a file, replacing existing contents. Here we use a fresh directory. **ls -l** displays permissions and other metadata.

Change permissions and test access

```
chmod 600 message.txt
ls -l message.txt
chmod 000 message.txt
cat message.txt
chmod 600 message.txt
```

In **600**, the owner receives read (4) plus write (2) permission. The group and others receive no permission. **000** removes the ordinary access permissions. The final command restores the owner's access. Use your ordinary account for this experiment.

Record and explain

Record the permission string after chmod 600:

-rw------- 1 riskgratis riskgratis 36 Oct 14 14:04 message.txt

What happened when you ran cat after chmod 000? Record the message or result.

I got permission denied: cat: message.txt: Permission denied

Did changing the permissions delete the file? How can you check after restoring access?

The file was not deleted: can be checked with 'ls' command

What does this experiment show about the operating system's role in protection?

It plays a critical role in access write and permissions. Preventing unauthorized users from accessing secure files

Keep this terminal open for Activity 6. The temporary directory may be cleaned later by the system, so it is not a location for long-term storage.

Trace requests to the kernel

Activity 6 of 6 | Optional 8 minutes | System calls and file descriptors

Continue in the directory from Activity 5 with message.txt readable again. This activity requires strace and an environment that permits tracing. If unavailable, record that fact and discuss the questions with your instructor.

Run this command

```
strace -e trace=openat,read,write,close cat message.txt
```

strace traces the program that follows it. **-e trace=...** filters the displayed system calls. The trace and cat's own output may appear together. Find message.txt among the startup and library-related operations.

openat opens a file, **read** reads from a file descriptor, **write** writes to one, and **close** closes it. A file descriptor is an integer identifying an open I/O object within a process. Exact calls and buffer sizes can vary.

Record your observations

Copy the trace line that opens message.txt, including its return value:

riskgratis@rgtech:/tmp/tmp.cF4hCXtxN6$ strace -e trace=openat,read,write,close cat message.txtopenat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = close(3)= 0openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6",O_RDONLY|O_CLOEXEC) = 3read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"..., 832) = 832close(3)= openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3close(3)= 0openat(AT_FDCWD, "message.txt", O_RDONLY) =3read(3, "Operating systems manage resourc"..., 131072) = 36w "Operating systems manage resourc"..., 36Operating systems manage resources.)= 36read(3, "", 131072)= 0close(3)= 0close(1)= 0close(2)= 0+++ exited with

Copy one read or write line. What do its arguments and return value tell you?

read(3, "Operating systems manage resourc"..., 131072) = 36

3 (Argument 1): The file descriptor being read from. "Operating systems..." (Argument 2): A preview of the content that was successfully read and placed into the program's memory buffer.131072 (Argument 3): The maximum number of bytes the cat program requested to read at once (the size of its allocated buffer). 36 (Return Value): The actual number of bytes successfully read from the file.

Explain

Is cat itself a system call? Explain the relationship between a command, a program and kernel services.

No, cat is a user-space program. It is not part of the kernel; rather, it uses system calls (like openat, read, and write) to ask the kernel to perform hardware and filesystem operations on its behalf.

Command: The instruction you type into the shell (e.g., cat message.txt). Program: The compiled, executable binary file on your storage drive that runs when you type the command. Kernel Services: The protected, privileged operations managed by the operating system.

Why might the trace mention files other than message.txt?

Before cat can actually open message.txt, the operating system must build its execution environment. The trace shows the dynamic linker preparing the program by reading /etc/ld.so.cache to locate dependencies, loading standard shared libraries like libc.so.6

Exit ticket

Choose one observation from today. State what it demonstrates about how an operating system works.

The strace output showing the cat command executing openat, read, and write.
This illustrates the principle of hardware abstraction. User-space programs cannot access files or hardware directly; they must request the operating system kernel to perform these privileged actions on their behalf using system calls.

State one thing your observations cannot establish about this computer.

These terminal outputs cannot establish the computer's actual physical hardware configuration, because the operating system might be running inside a virtual machine that hides the physical host's reality.

Before finishing: Complete each observation field, restore message.txt permissions to 600, and end any sleep process you started. Mark unavailable activities and the reason. Share your worksheet as instructed by your lecturer.